Patrones de diseño de comportamiento

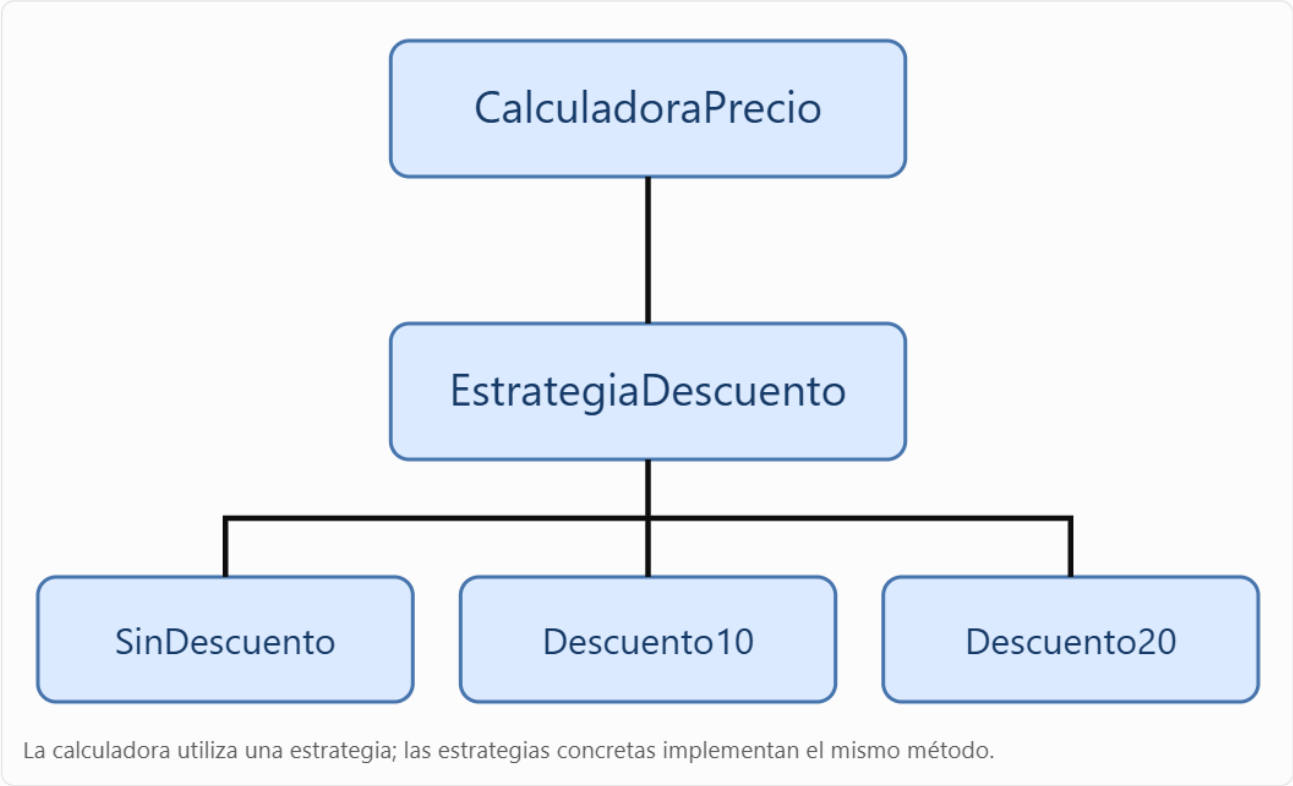
Objetivo: comprender qué problema resuelve cada patrón, identificar dónde aparece el polimorfismo y practicar con ejemplos pequeños, sin interfaz gráfica.

1. Strategy (Estrategia)

¿Qué problema resuelve?

Cuando una operación admite varias maneras de calcularse, Strategy permite encapsular cada algoritmo en una clase intercambiable. La clase que utiliza el algoritmo no necesita conocer cuál es la estrategia concreta.

Ejemplo: una calculadora de precios puede aplicar distintos descuentos. En lugar de utilizar if/elif para cada tipo de descuento, recibe un objeto estrategia.



La calculadora utiliza una estrategia; las estrategias concretas implementan el mismo método.

Ejemplo resuelto

```
from abc import ABC, abstractmethod

class EstrategiaDescuento(ABC):
    @abstractmethod
    def calcular(self, precio):
        pass

class SinDescuento(EstrategiaDescuento):
    def calcular(self, precio):
        return precio

class Descuento10(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.90
```

```
class Descuento20(EstrategiaDescuento):
    def calcular(self, precio):
        return precio * 0.80

class CalculadoraPrecio:
    def __init__(self, estrategia):
        self.estrategia = estrategia

    def calcular_precio(self, precio):
        return self.estrategia.calcular(precio)

    def cambiar_estrategia(self, estrategia):
        self.estrategia = estrategia

calculadora = CalculadoraPrecio(Descuento10())
print(calculadora.calcular_precio(10000))  # 9000.0
calculadora.cambiar_estrategia(Descuento20())
print(calculadora.calcular_precio(10000))  # 8000.0
```

La instrucción self.estrategia.calcular(precio) usa polimorfismo: ejecuta el método de la estrategia recibida, sin preguntar de qué clase concreta se trata.

Ejercicio 1: costos de envío

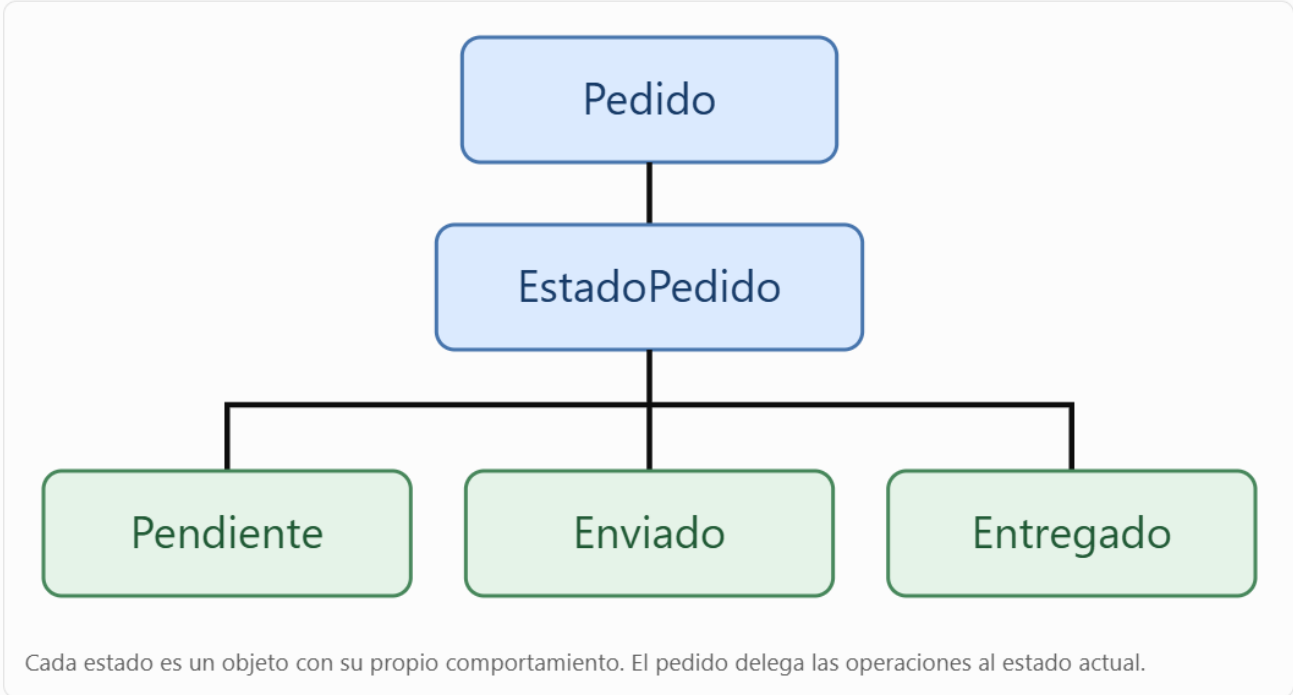
Crear EstrategiaEnvio (abstracta, con calcular_costo(peso)), EnvioNormal ($1.000/kg), EnvioExpress ($1.500/kg + $2.000 fijos) y EnvioEconomico ($800/kg). Crear CalculadoraEnvios que permita cambiar la estrategia y calcular el costo de un paquete de 5 kg con las tres modalidades.

2. State (Estado)

¿Qué problema resuelve?

State permite que un objeto modifique su comportamiento según su estado actual. Cada estado se representa mediante un objeto con sus propias operaciones. El objeto principal delega la acción al estado vigente.

Ejemplo: un pedido pasa por Pendiente → Enviado → Entregado. Avanzar significa algo diferente en cada etapa.



Cada estado es un objeto con su propio comportamiento. El pedido delega las operaciones al estado actual.

Ejemplo resuelto

```
from abc import ABC, abstractmethod

class EstadoPedido(ABC):
    @abstractmethod
    def avanzar(self, pedido):
        pass

class Pendiente(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido fue enviado")
        pedido.cambiar_estado(Enviado())

class Enviado(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido fue entregado")
        pedido.cambiar_estado(Entregado())

class Entregado(EstadoPedido):
    def avanzar(self, pedido):
        print("El pedido ya fue entregado")

class Pedido:
    def __init__(self):
        self.estado = Pendiente()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def avanzar(self):
        self.estado.avanzar(self)

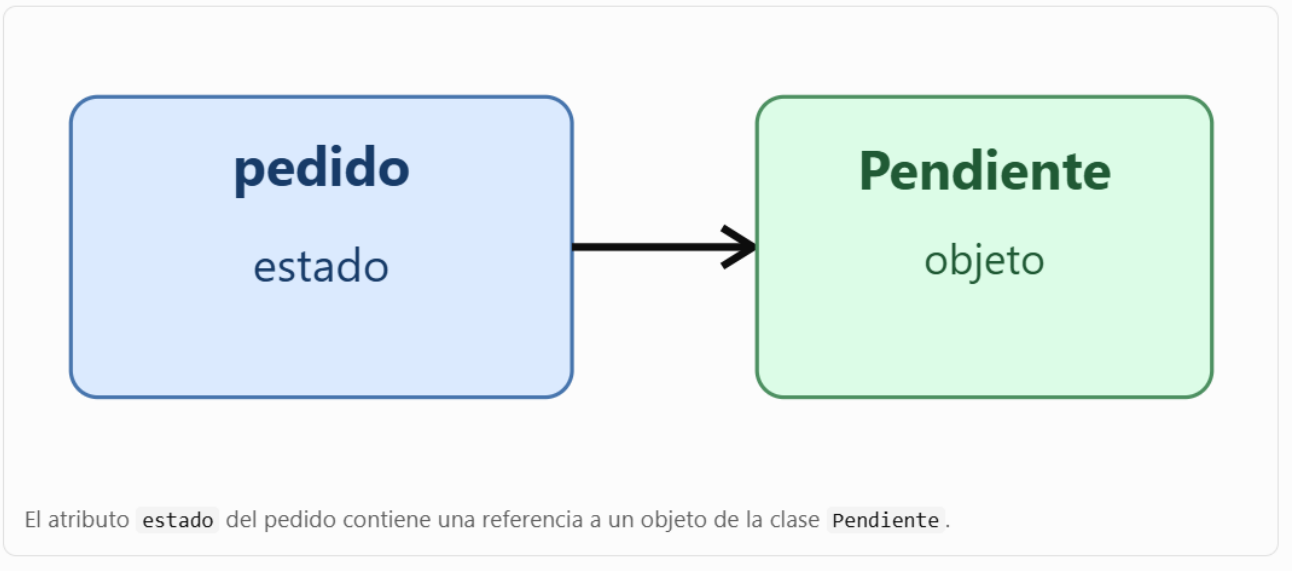
pedido = Pedido()
pedido.avanzar()
pedido.avanzar()
pedido.avanzar()
```

Explicación detallada: ¿qué hace Pendiente()?

Aunque la clase Pendiente no declare __init__(), Python puede crear una instancia con Pendiente(). En este caso utiliza el inicializador heredado de object. La expresión Pendiente() produce un objeto, no el texto "Pendiente".

```
estado = Pendiente()
print(type(estado))  # <class '__main__.Pendiente'>
```

Cuando se ejecuta pedido = Pedido(), el constructor de Pedido crea un objeto Pendiente y guarda su referencia en self.estado. No se guarda un nombre de estado: se guarda un objeto.



El atributo `estado` del pedido contiene una referencia a un objeto de la clase `Pendiente`.

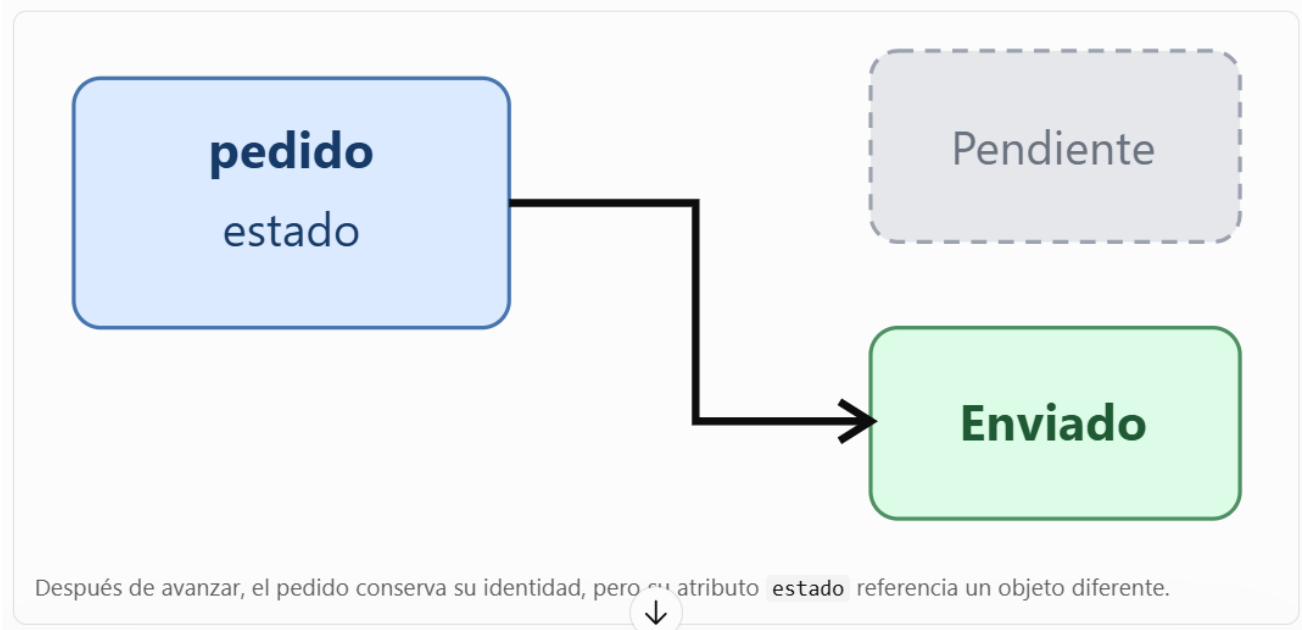
¿Qué significa self.estado.avanzar(self)?

Hay dos objetos distintos. En Pedido.avanzar(), self representa el Pedido. Su atributo estado referencia inicialmente un Pendiente. Por eso self.estado.avanzar(self) llama al método avanzar del objeto Pendiente y le pasa el Pedido como argumento.

Dentro de Pendiente.avanzar(self, pedido), el primer self es el objeto Pendiente, mientras que pedido es el objeto Pedido recibido. Se pasa el pedido porque el estado necesita modificar su estado actual.

```
# En Pedido:
def avanzar(self):
    self.estado.avanzar(self)

# En Pendiente:
def avanzar(self, pedido):
    print("El pedido fue enviado")
    pedido.cambiar_estado(Enviado())
```



¿Qué hace cambiar_estado()?

Enviado() crea una instancia de Enviado. cambiar_estado() reemplaza la referencia guardada en pedido.estado. El objeto Pedido sigue siendo el mismo, pero ahora delegará su próxima llamada a Enviado.avanzar().

Importante: avanzar() no devuelve un nuevo estado mediante return. Su efecto es modificar el atributo estado del Pedido. Como no tiene return explícito, el método devuelve None.

Seguimiento paso a paso

```
class Pedido:
    def __init__(self):
        self.estado = Pendiente()

    def cambiar_estado(self, nuevo_estado):
        self.estado = nuevo_estado

    def avanzar(self):
        print("Antes:", type(self.estado).__name__)
        self.estado.avanzar(self)
        print("Después:", type(self.estado).__name__)

pedido = Pedido()
pedido.avanzar()  # Pendiente -> Enviado
pedido.avanzar()  # Enviado -> Entregado
pedido.avanzar()  # permanece Entregado
```

Secuencia conceptual: Pedido(Pendiente) → avanzar() → Pedido(Enviado) → avanzar() → Pedido(Entregado). El polimorfismo permite usar siempre la misma llamada: self.estado.avanzar(self).

¿Por qué no usar strings y condicionales?

Con estado = "pendiente" se necesitarían if/elif para decidir qué hacer. Con State, cada clase concentra el comportamiento de su estado y puede decidir cuándo corresponde una transición. Para ejemplos pequeños, los condicionales también funcionan; el patrón resulta especialmente útil cuando crecen los estados y las operaciones.

Ejercicio 2: máquina expendedora

Implementar MaquinaExpendedora con los estados SinDinero, ConDinero y SinStock. Operaciones: ingresar_dinero(), comprar() y reponer_stock(). Cada estado debe decidir qué operaciones permite y qué transición produce. Definir explícitamente qué ocurre cuando no hay stock o se intenta comprar sin dinero.

3. Iterator (Iterador)

¿Qué problema resuelve?

Iterator permite recorrer una colección sin exponer cómo almacena sus elementos. En Python, los ciclos for utilizan el protocolo de iteración: iter() obtiene un iterador y next() obtiene el elemento siguiente, hasta StopIteration.

En Python ya existen iteradores, que se utilizan automáticamente en los ciclos for.

Para comprender el patrón construiremos un iterador explícito, aunque las listas de Python ya incluyen sus propios iteradores.

Ejemplo resuelto

```
class IteradorLibros:
    def __init__(self, libros):
        self.libros = libros
        self.posicion = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.posicion >= len(self.libros):
            raise StopIteration
        libro = self.libros[self.posicion]
        self.posicion += 1
        return libro

class Biblioteca:
    def __init__(self):
        self.libros = []

    def agregar_libro(self, titulo):
        self.libros.append(titulo)

    def __iter__(self):
        return IteradorLibros(self.libros)

biblioteca = Biblioteca()
biblioteca.agregar_libro("El Principito")
biblioteca.agregar_libro("Don Quijote")
biblioteca.agregar_libro("Martín Fierro")

for libro in biblioteca:
    print(libro)
```

El for llama a biblioteca.__iter__(), que crea un IteradorLibros. Luego invoca __next__() sucesivamente. Cuando no quedan libros, StopIteration indica el fin del recorrido.

Ejercicio 3: curso y alumnos

Crear Alumno(nombre, nota), Curso (almacena alumnos) e IteradorAlumnos. Permitir recorrer con for alumno in curso. Incorporar un segundo iterador que entregue solamente los alumnos con nota mayor o igual a 6. El programa principal no debe recorrer directamente la lista interna del curso.

4. Comparación y cierre

Patrón	Pregunta que responde	Idea clave
Strategy	¿Qué algoritmo utilizo?	Intercambiar estrategias
State	¿Qué puede hacer el objeto ahora?	Comportamiento y transición de estados
Iterator	¿Cómo recorro los elementos?	Recorrido encapsulado

State y Strategy pueden parecer similares: ambos delegan en objetos con una interfaz común. La diferencia principal es la intención: Strategy permite seleccionar un algoritmo; State representa la evolución del comportamiento de un objeto a través de sus estados.

Preguntas de repaso

- ¿En qué instrucción de Strategy aparece el polimorfismo?
- ¿Por qué Pendiente() funciona sin un __init__ propio?
- En self.estado.avanzar(self), ¿qué representa cada self?
- ¿Qué cambia en Pedido cuando pasa de Pendiente a Enviado?
- ¿Qué diferencia hay entre __iter__() y __next__()?
- ¿En qué situaciones preferirías un patrón en lugar de una solución sencilla con if o for?